

Modular Hangman Game

Ashfaql Alam
Student ID: 253374642
Dept. of Electrical and
Computer Engineering, North
South University
ashfaql.alam.253@northsout
h.edu

Intisar Binthay Fauzul
Student ID: 2533052642
Dept. of Electrical and
Computer Engineering, North
South University
intisar.fauzul.253@northsout
h.edu

Rafiha Saleha
Student ID: 2531742042
Dept. of Electrical and
Computer Engineering, North
South University
rafiha.saleha.253@northsouth
.edu

Syed Sadat Hasan
Student ID: 2533781042
Dept. of Electrical and
Computer Engineering, North
South University
sadat.hasan.253@northsouth.
edu

Abstract

This project's primary goal was to build a working version of the classic Hangman game, built entirely using the C programming language. The game we developed is a straightforward, text-based game which provided an excellent opportunity to use and practice foundational concepts of C programming as well as programming in general.

The game includes several key elements. The system dynamically pulls words for the game from a separate text file using file handling. The interface provides a clear visual representation of the hangman figure using simple ASCII art, which updates every time the player makes an incorrect guess. The program also includes solid input validation for smooth operation and running of the game,

correctly tracking remaining attempts and the progress of the secret word. This project showcases the effective use of C to develop clean, self-contained applications directly within the console environment. The modular nature of the project also allows clean implementation, easier bug fixes, and future upgradeability of the game.

I. INTRODUCTION

Hangman is a classic word-guessing game in which a player attempts to reveal a hidden word by guessing individual letters within a limited number of attempts. Despite its simplicity, the game requires the implementation of several important programming concepts, making it an ideal project for learning the

fundamental concepts of C programming language and software development in general.

This project aims to develop a console-based Hangman game that is modular, robust, and extensible. The game uses arrays and strings to manage word data, functions to encapsulate logic, and file handling to store both the word list and player scores. By structuring the program into multiple source files and functions, the project encourages good coding practices and teamwork.

This paper describes the problem statement, objectives, system architecture, algorithms, implementation details, testing strategy, and possible future enhancements.

II. PROBLEM STATEMENTS AND OBJECTIVES

A. Problem Statement

The primary challenge addressed by this project is the design of a text-based Hangman game that is both functional and well-structured. The system must allow random word selection from an external source, prevent duplicate guesses, provide feedback to the player, and track scores persistently across multiple game sessions. Additionally, the project must be implemented using the C programming language and documented using the IEEE conference format.

B. Objectives

The main objective of this project are:

- Implementing a fully functioning Hangman game using C programming language.

- Applying modular programming techniques using functions and separate header files.
- Using file input/output for word storage and score persistence.
- Validating user input and handling edge cases.
- Designing a scoring system that updates player performance.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Overall Architectures

The Hangman game's system is divided into multiple components, each carrying out a specific task. The game's core logic is based on a set of reusable functions, with the main function coordinating the execution of the program. External text files are used for storing relevant data.

System Components:

<i>File Name</i>	<i>Purpose</i>
main.c	Program entry point and initialization
game.c	Core game logic and helper functions
game.h	Game logic and helper function declarations and share definitions
player.c	Functions to load and save player

	information such as scores
player.h	Player information function declarations and shared definitions
words.txt	Text file to store the list of words for the game

processGuess()	Processes player input, determines if player input is correct, wrong or repeated.
checkWin()	Checks if all letters of the secret word have been guessed correctly and returns 1 if so.
checkifGuessed()	Checks if a letter has been already guessed

B. Functional Decomposition

Key Functions:

<i>Function Name</i>	<i>Function Purpose</i>
playHangman()	The core function to bring everything together, controls the main game loop and integrates all its components.
chooseWord()	Chooses a random word from the list to be the secret word the player needs to guess.
updateProgress()	Initiates the game and updates its progress based on player input.
playerGuess()	Collects player input and trims it to only accept valid (character) inputs.

Other Functions:

<i>Function Name</i>	<i>Function Purpose</i>
getPlayerName()	Takes player name as input
savePlayerScore()	Saves player score for each individual player name
freeWordList()	Frees allocated memory for the word list
showHangman()	Draws ASCII art of the hangman figure based on the number of attempts the player has left
displayProgress()	Displays the current progress of the game

This style of modular design improves the readability and maintainability of the code for the game, and allows for easy identification of problems or bugs as well as the implementation of new features. The game's initialization using these functions can be seen in Fig. 1.

```

+-----+
| WELCOME TO THE HANGMAN GAME! |
+-----+
Enter your name: Bob
Hello, Bob! Let's play Hangman!
Loaded 50 words from dictionary.

-----
NEW ROUND START!!!
-----
The word has 7 letters. Good luck!

|_____|
|      |
|      |
|      |
|_____|

Word:
Guesses left: 6
Enter a letter:

```

Fig. 1. Hangman game initialization, loading words from directory and displaying the number of letters of the word and the number of tries the player has left.

IV. ALGORITHMS AND IMPLEMENTATION

A. Hangman Game Algorithm

Main Game Loop:

- Initialize game variables and arrays
- Load a random word from word.txt
- Reveal a proportional number of random letters
- While attempts remain:

- Display hangman figure
- Display the current progress of the game i.e. how much of the secret word has been revealed
- Read player guess
- Process player guess and update the state of the game accordingly.
- Check win condition of the game
- Declare game won if all letters are guessed by the player, or game lost if the player runs out of attempts.
- Update player score and display the result

The logical loop followed by the main game is shown in Fig. 2.

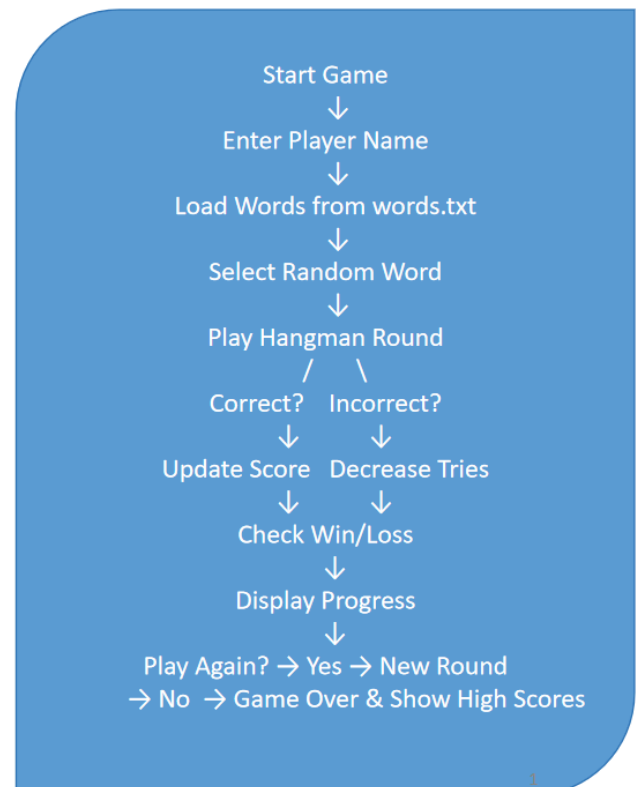


Fig. 2. Hangman game logical loop

B. Guess Processing

- The guess processing logic checks if a guess is correct, wrong or duplicate and proceeds accordingly.
- On correct guess, it updates the displayed word, revealing the correctly guessed letter in every blank position it was in.
- On a wrong guess, it reduces the number of attempts left by the player
- On a duplicate guess, it informs the player that the guess is a duplicate and prompts for a different input without updating the displayed word or reducing the number of attempts left.
- Ensures that only valid inputs are processed.

V. FILE HANDLING AND DATA PERSISTENCE

A critical aspect of the project is file handling. The word list is stored in *words.txt* with one word per line for ease of parsing. Using external files allows for the system to be easily updated and the list of words to be changed without having to modify the source code.

VI. TESTING AND RESULTS

The program was tested using multiple test cases to ensure that it is functioning as intended. Thus the robustness and correctness of the game was ensured. The test cases included entering correct guesses, incorrect guesses, duplicate guesses, invalid inputs,

winning and losing the game as well as starting different games to ensure that the word selection truly is randomized.

<i>Test Case</i>	<i>Input</i>	<i>Expected Output</i>	<i>Result</i>
Correct guess	'a'	Letter revealed	Passed
Incorrect guess	'x'	Tries reduced by 1, updated hangman	Passed
Duplicate guess	'a' again	Warning message, no change	Passed
Invalid input	'#'	Re-prompt	Passed
Complete word	Correct guesses	Win message, score update	Passed
Game lost	Incorrect guesses until 0 tries left	Complete hangman figure, lose message	Passed
New Game	'Y' after completing a game	Different, randomized secret word	Passed

REFERENCES

- [1] [cppreference.com](https://en.cppreference.com/w/c), “C Standard Library Reference,”
<https://en.cppreference.com/w/c>
- [2] GeeksforGeeks, “Hangman Game in C,”
<https://www.geeksforgeeks.org/c-program-hangman-game/>
- [3] Programiz, “#29: C File Handling | [2025] C Programming for Beginners,” YouTube,
<https://www.youtube.com/watch?v=MQIF-WMUOL8>
- [4] OpenAI, “ChatGPT: Large Language Model for Code and Documentation Assistance,”
<https://chat.openai.com>